5

Searching in Dynamic Data

In data structures, a dynamic environment refers to a setting in which the dataset is frequently changing due to insertions, deletions, or updates, and where the size or ordering of the data may not remain fixed over time. Unlike static environments—where data remains relatively stable and preprocessed—dynamic environments require algorithms that can efficiently adapt to continuous modifications. Comparison-based searching techniques such as linear search and binary search have significant limitations in such contexts. Linear search, while simple and adaptable, becomes inefficient as data grows because each search may require scanning many elements $O(n)$, which is costly when searches are frequent. Binary search improves search time to $O(\log n)$, but it requires the data to remain sorted; maintaining sorted order in a dynamic dataset introduces additional overhead. Insertions and deletions in sorted arrays often require shifting elements $O(n)$, making updates expensive and reducing overall efficiency. Furthermore, comparison-based methods rely solely on element-to-element comparisons, limiting their scalability when rapid, real-time lookups are needed. The main challenges in dynamic environments therefore include maintaining performance under frequent updates, balancing search and modification costs, preserving structural invariants (such as sorted order), and ensuring scalability as the dataset grows. These limitations motivate the use of alternative structures such as hash tables and balanced trees, which are specifically designed to handle dynamic data more efficiently.

5.1 Hash Table

A hash table is a data structure that stores data as key–value pairs and provides very fast average-case performance for insertion, deletion, and search operations. It works by using a hash function to transform a key (such as a string, integer, or object) into an integer index that corresponds to a position in an underlying array. Instead of scanning through elements one by one (as in linear search) or relying on sorted order (as in binary search), a hash table computes where the data should be stored and accesses it directly. This is why hash tables typically achieve $O(1)$ average time complexity for lookup, insertion, and deletion.

Mention that the hash table discussion assumes internal storage and not external storage.

Hash Tables in Modern Programming Languages
Hash tables are referred to by different names across programming languages—for example, `HashMap` in Rust and Java; `std::unordered_map` in C++; `dict` in Python; `map` in Go, as few examples.

Figure 5.1 illustrates how hash tables store data in memory using an underlying array as the primary storage structure. This array consists of a fixed number of slots, commonly referred to as buckets, and each bucket corresponds to an index derived through a two-step process involving a hash function and a compression function.

When a key is inserted, the hash function first transforms the key into a large integer value using a specified hashing algorithm. This integer hash value is then passed to the compression function, which maps it into a valid array index within the bounds of the underlying array. Once the index is computed, the key–value pair is stored at the corresponding location in the contiguous array structure, conceptually labeled as $A_1, A_2, A_3, \ldots, A_n$. The colored cells in the figure visually represent how different keys are mapped to different buckets. Importantly, this placement is determined by the computed hash and compression processes rather than by the order of insertion, thereby enabling efficient and direct access to stored elements.

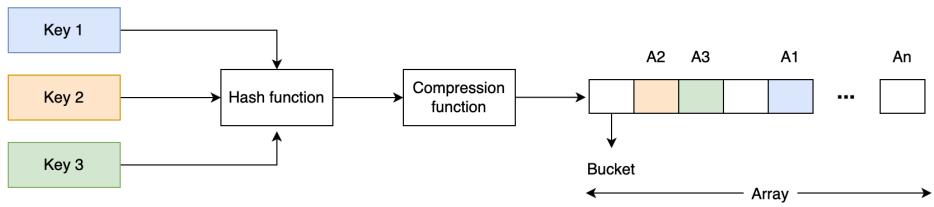

FIGURE 5.1 *Hash Table*

The following example illustrates how a key is mapped to an index within an array. Its purpose is not to present the full theoretical framework of hashing, but rather to demonstrate the basic operational mechanism of a hash implementation and provide the reader with an initial intuitive understanding. Hashing is a conceptually rich and mathematically nuanced topic; therefore, many of its deeper properties—such as collision behavior, distribution quality, robustness considerations, and performance analysis—will be examined systematically in the subsequent sections.

Polynomial rolling hash

Suppose, the key is "CAT". The ASCII value of CAT is 67, 65 and 84 respectively. For simplicity, a polynomial rolling hash function is used to compute the index. A polynomial rolling hash is a string hashing technique that treats a string like a polynomial and evaluates it at a chosen base number to produce a numeric hash value. It is called "polynomial" because each character contributes to the hash as a term in a polynomial expression, and "rolling" because the hash can be efficiently updated when sliding over substrings (commonly used in algorithms like Rabin–Karp). The hash function will be discussed further in next section.

The hash can be calculated using the following equation.

$$\text{hash}(s) = c_1 \cdot p^{n-1} + c_2 \cdot p^{n-2} + \cdots + c_n \cdot p^0 \tag{5.1}$$

where n is the number of characters in the string, s is the input string of length n, c_i is the numeric value (e.g., ASCII or Unicode) of the ith character, p is a chosen base (commonly a small prime such as 31 or 37), p^k is the positional weight assigned to each character and hash(s) is the resulting integer hash value.

Use Equation 5.1, the hash value of CAT can be computed as follows:

$$\begin{aligned}\text{hash(CAT)} &= c_1 \times 31^2 + c_2 \times 31^1 + c_3 \times 31^0 \\ &= 67 \times 31^2 + 65 \times 31^1 + 84 \times 31^0 \\ &= 66486\end{aligned}$$

The prime number used in a polynomial rolling hash is a deliberate design choice intended to promote uniform distribution of hash values and mitigate structured collisions. Its selection is grounded in number-theoretic principles and empirical performance considerations rather than being derived from the input string itself. For example, Java's String.hashCode() employs a polynomial hash with a base of 31, an odd prime chosen to balance computational efficiency and distribution quality [2]. Similarly, in algorithmic and competitive programming contexts, primes such as 31 or 53 are commonly recommended, with the specific choice often influenced by the size of the character set. These primes help achieve effective dispersion of hash values while maintaining efficient arithmetic operations.

The next step is to apply a compression function to the computed hash value in order to obtain a valid array index. Lets assume that the division method is used.

$$\text{index} = \text{hash}(key) \text{ mod array_length} \tag{5.2}$$

where array_length is the total number of buckets in the hash table and the mod is the modulo operator (returns the remainder). The modulo operation guarantees that $0 \leq \text{index} < \text{array_length}$, ensuring that the computed index always falls within the valid bounds of the underlying array. When a hash table is created, its implementation allocates a contiguous array consisting of x buckets. Each bucket represents a storage slot within the table. The value of x is implementation-dependent and varies across programming languages and standard library implementations.

Terminology Clarification

In many textbooks, the term table length is used to denote the number of buckets in a hash table, which corresponds directly to the length of the underlying array. For the sake of clarity and terminological uniformity, we will use the term array length throughout this discussion.

For this example, suppose that the array contains 16 buckets (i.e., array_length $= 16$) and that the previously computed hash value of CAT is 66486. By applying Equation 5.2, we calculate 66486 mod 16, which produces an index value of 6. This index identifies the specific bucket within the underlying array where the associated key–value pair is stored.

In most hash table implementations, keys are treated as case-sensitive, particularly when the key type is a string. Consequently, the strings "CAT" and "cat" are considered distinct keys because their character encodings differ at the byte level. It is important to note that case sensitivity is not an inherent property of the hash table data structure itself; rather, it is determined by the key's data type and the equality comparison mechanism defined by the programming language or implementation.

The above example illustrates the basic idea behind hash and compression functions in simple terms. In the next section, we will examine their design in greater depth and explore different approaches to constructing hash and compression functions.

5.2 Principles of Hash Function Design

In this section, we examine hash functions as they are used in hash tables for efficient data storage and retrieval. The emphasis is on properties such as uniform key distribution, minimization of collisions, and computational efficiency, all of which directly influence the average-case performance of fundamental operations.

This discussion does not address cryptographic hash functions (e.g., SHA-256), which are designed to satisfy security-oriented properties such as strong collision resistance, preimage resistance, and one-way behavior.

5.2.1 Deterministic mapping

A hash function used in a hash table is formally defined as

$$h : \mathcal{K} \to \mathbb{Z} \tag{5.3}$$

Here, $\mathcal{K}$ denotes the set of keys and $\mathbb{Z}$ denotes the set of integers. By the definition of a mathematical function, each key $k \in \mathcal{K}$ is associated with a unique integer $h(k) \in \mathbb{Z}$. Therefore, the mapping defined in Equation 5.3 is deterministic: repeated evaluations of h on the same key must produce the same integer value. Formally, using the universal quantifier symbol $\forall$ (read as "for all"),

$$\forall k \in \mathcal{K}, \quad h(k) \text{ is fixed} \tag{5.4}$$

The necessity of determinism follows from the operational behavior of a hash table. During insertion at time t_1, the storage location of a key is computed from $h(k)$. During lookup at time t_2, the same computation is performed. Correct retrieval therefore requires

$$h_{t_1}(k) = h_{t_2}(k) \tag{5.5}$$

Equation 5.5 follows directly from the definition of the function in Equation 5.3. If the hash function were allowed to vary between evaluations, the computed bucket

during lookup could differ from the bucket used during insertion, thereby violating correctness.

Determinism is also essential for referential consistency during structural operations such as table resizing. When the table capacity changes from m to $\bar{m}$, only the compression function is modified. The underlying hash value defined in Equation 5.4 must remain invariant so that reinsertion preserves the identity of each key.

Therefore, determinism is a fundamental requirement for correctness, stability, and referential consistency in hash table implementations.

5.2.2 Uniform Distribution

The performance of a hash table depends fundamentally on how evenly keys are distributed across its m buckets. Let

$$h : \mathcal{K} \to \{0, 1, \dots, m-1\} \tag{5.6}$$

be a hash function that maps keys from the universe $\mathcal{K}$ into an array of size m. The set $\{0, 1, \dots, m-1\}$ represents all valid bucket indices of the table. For efficient performance, it is desirable that each bucket be equally likely to receive any given key. Formally, this requirement is expressed as

$$\Pr[h(k) = i] = \frac{1}{m}, \qquad \forall i \in \{0, 1, \dots, m-1\}. \tag{5.7}$$

The Equation 5.7 states that the probability that a key k is mapped to a specific bucket i is exactly $1/m$. Since there are m buckets, each bucket receives an equal fraction of the total probability mass. Consequently, the probabilities sum to one.

$$\sum_{i=0}^{m-1} \frac{1}{m} = 1 \tag{5.8}$$

This uniformity condition forms the basis of the analytical model known as Simple Uniform Hashing (SUH), which assumes that each key is hashed independently and uniformly at random into one of the m buckets. Although practical hash functions are deterministic, SUH provides a probabilistic abstraction that enables rigorous average-case analysis.

Simple uniform hash

Suppose n keys are inserted into the table. For a fixed bucket i, define the random variable, X_i to represent the number of keys that are mapped to bucket i by the hash function. In other words, X_i counts how many of the n inserted keys hash to the specific bucket indexed by i.

Under SUH, each key independently maps to bucket i with probability $1/m$. Therefore, the number of keys in bucket i follows a binomial distribution:

$$X_i \sim \text{Binomial}(n, 1/m) \tag{5.9}$$

This distribution arises because there are n independent trials (one for each key), and each trial results in a "success" (the key hashes to bucket i) with probability $1/m$.

The expected number of keys in bucket i is given by the expectation of a binomial random variable $\mathbb{E}[X_i] = n \cdot \frac{1}{m}$. This simplifies to $\mathbb{E}[X_i] = \frac{n}{m}$. The quantity

$$\alpha = \frac{n}{m} \tag{5.10}$$

Load factor

is called the load factor, representing the average number of keys n stored per bucket m. Thus, $\mathbb{E}[X_i] = \alpha$. The load factor is a quantitative measure of how full a hash table is. It represents the average number of keys stored per bucket. There are three cases:
Case 1: If $\alpha = 1$, then on average each bucket contains one key.
Case 2: If $\alpha < 1$, the table is relatively sparse.
Case 3: If $\alpha > 1$, buckets (in separate chaining) contain multiple keys on average.

5.2.3 Low Collision Probability

A fundamental objective in the design of a hash function for hash tables is to minimize the probability of collisions. A collision occurs when two distinct keys $k_1, k_2 \in \mathcal{K}$, with $k_1 \neq k_2$, are mapped to the same bucket $h(k_1) = h(k_2)$.

Pigeonhole principle

Before examining collision behavior in hash tables, it is instructive to appeal to the Pigeonhole Principle as a fundamental counting argument. This principle provides a rigorous mathematical explanation for why collisions are unavoidable in hashing. By interpreting keys as objects and buckets as containers, the principle implies that whenever the number of keys exceeds—or is comparable to—the number of buckets, at least one bucket must contain more than one key.

In the setting of a hash table with m buckets storing n keys, collisions are therefore inevitable when $n \geq m$. Even in cases where $n \leq m$, collisions may still arise depending on the distribution induced by the hash function. Consequently, the objective in hash function design is not to eliminate collisions entirely, which is impossible in general, but to ensure that they occur with low probability and without systematic bias. The Pigeonhole Principle thus provides the theoretical foundation for analyzing collision probability and justifies the necessity of collision resolution mechanisms in hashing.

Probabilistic Model

Under the assumption of simple uniform hashing, each key is assumed to be mapped independently and with equal probability to any of the m buckets, as expressed in Equation 5.7.

For two distinct keys k_1 and k_2, the probability that they collide is:

$$\Pr[h(k_1) = h(k_2)] = \frac{1}{m} \tag{5.11}$$

If n keys are inserted into the table, there are

$$\binom{n}{2} = \frac{n(n-1)}{2} \tag{5.12}$$

distinct pairs of keys.

The expression $\binom{n}{2}$, read as "n choose 2," denotes a binomial coefficient representing the number of distinct unordered pairs that can be selected from a set of n keys. More generally, the binomial coefficient is defined as

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

where $n!$ denotes the factorial of n.

Here $r = 2$, because a collision involves exactly two distinct keys, we obtain

$$\binom{n}{2} = \frac{n!}{2!(n-2)!} \tag{5.13}$$

Simplifying the factorial expression yields

$$\binom{n}{2} = \frac{n(n-1)}{2} \tag{5.14}$$

Equation 5.14 counts how many different pairs can be formed from n keys when the order does not matter. For example, if you have 4 keys, you can form 6 pairs like (Key1-Key2), (Key1-Key3), (Key1-Key4), (Key2-Key3), (Key2-Key4), and (Key3-Key4). In other words, $\binom{n}{2}$ simply tells us how many different key-pairs exist that could potentially collide.

To compute the expected total number of collisions, multiply number of possible pairs and the probability that any given pair collides.

$$\begin{aligned}\mathbb{E}[\text{colliding pairs}] &= \binom{n}{2} \cdot \frac{1}{m} \\ &= \frac{n(n-1)}{2} \cdot \frac{1}{m} \\ &= \frac{n(n-1)}{2m}\end{aligned} \tag{5.15}$$

Effect of Increasing the Number of Buckets
Equation 5.15 shows that, for fixed n, the expected number of colliding pairs decreases as m increases, since the expression is inversely proportional to m. This conclusion holds under the assumption of simple uniform hashing. If the hash function does not distribute keys uniformly, increasing m alone may not yield a corresponding reduction in collisions.

5.2.4 *Avalanche Effect*

The avalanche effect refers to the property of a hash function whereby a small change in the input key produces a large and seemingly unpredictable change in the resulting hash value. In the context of hash tables, this property promotes effective key dispersion and helps prevent clustering caused by structurally similar inputs.

If the change in the hash value is also small when the input is slightly modified, the outputs become correlated. In such cases, similar keys may map to nearby or identical buckets, leading to non-uniform distribution. This correlation increases the likelihood of clustering, degrades average-case performance, and may cause the hash table to deviate from its expected $O(1)$ behavior. Therefore, insufficient sensitivity to small input variations is indicative of a weak hash function for practical hashing applications.

Let

$$h : \mathcal{K} \to \{0,1\}^w \tag{5.16}$$

be a hash function whose output is represented as a binary string of length w bits.

$$k_1, k_2 \in \mathcal{K}, \quad k_1 \neq k_2, \tag{5.17}$$

such that k_2 differs only minimally from k_1. For example, a single character change in a string, a single bit flip in an integer, and a small increment in a numeric key. To measure the difference between their hash outputs, we use the Hamming distance. The Hamming distance is a measure of the difference between two strings of equal

Hamming distance

length. It is defined as the number of positions at which the corresponding symbols differ.

Formally, let

$$x = (x_1, x_2, \dots, x_w), \quad y = (y_1, y_2, \dots, y_w) \tag{5.18}$$

where x and y denote two strings of length w over the same alphabet (commonly binary). The Hamming distance between x and y is defined as

$$d_H(x, y) = \#\{i \in \{1, \dots, w\} : x_i \neq y_i\} \tag{5.19}$$

Where, $d_H(x, y)$ is the number of positions at which the two strings differ, $\{1, \dots, w\}$ is the set of all index positions in the strings, $x_i \neq y_i$ are the two strings differ at position i, $\{\, i \in \{1, \dots, w\} : x_i \neq y_i \,\}$ represents the set of all positions where the two strings differ and $\#\{\dots\}$ denotes the cardinality of a set, i.e., counts how many elements are in that set.

Using Equation 5.19, the Hamming distance of k_1 and k_2 can be represented by

$$d_H(h(k_1), h(k_2))$$

Now, determine the expected Hamming distance between the two outputs.

Assume the hash output has w bits. For a small change in the input, each output bit flips with probability $\frac{1}{2}$ and each bit flip is approximately independent.

Let X_i be the indicator variable

$$X_i = \begin{cases} 1 & \text{if bit } i \text{ differs,} \\ 0 & \text{otherwise.} \end{cases}$$

Then, the probability that the i-th bit differs between the two hash outputs is one-half.

$$\Pr(X_i = 1) = \frac{1}{2}$$

The Hamming distance is the total number of differing bits:

$$d_H = \sum_{i=1}^{w} X_i$$

Since each X_i is Bernoulli with probability $\frac{1}{2}$,

$$\mathbb{E}[X_i] = \frac{1}{2}.$$

By linearity of expectation, the expected Hamming distance between the two outputs will be:

$$\mathbb{E}[d_H] = \sum_{i=1}^{w} \mathbb{E}[X_i] = w \cdot \frac{1}{2} = \frac{w}{2} \tag{5.20}$$

Equation 5.20 explains that each bit in the hash output has a 50% chance of changing when the input is slightly modified. Since there are w total bits and each contributes an average of $\frac{1}{2}$ difference, the total expected number of differing bits is $w \times \frac{1}{2} = \frac{w}{2}$. In simple terms, if each bit behaves like a fair coin flip, then about half of the bits will differ on average.

The explanation below will help readers better understand the concept.

Statistical Justification for the $\frac{w}{2}$ Expectation: The expected Hamming distance is not w because flipping all bits would produce a perfectly inverted and therefore highly structured output, indicating deterministic behavior rather than randomness. Conversely, if only a few bits change, the outputs remain strongly correlated, reflecting poor mixing. The most desirable behavior occurs when each bit flips independently with probability $\frac{1}{2}$, yielding maximal uncertainty and entropy. In this case, the hash output changes in a balanced and statistically unpredictable manner, which promotes effective dispersion.

Why Not Exactly $\frac{w}{2}$? The Hamming distance is a random variable with mean $\frac{w}{2}$ and variance $\frac{w}{4}$. Consequently, the value of d_H fluctuates around $\frac{w}{2}$; some instances will exhibit slightly fewer differing bits, while others will exhibit slightly more. Therefore, $\frac{w}{2}$ represents an expected value rather than an exact deterministic outcome.

5.2.5 Robustness to Structured Input Patterns

The theoretical analysis of hash functions often assumes that inputs are uniformly random and mutually independent. Under this assumption, performance guarantees—such as expected constant-time lookup in hash tables—are derived from probabilistic models like the Simple Uniform Hashing Assumption (SUH). However, real-world inputs are rarely random. They frequently exhibit structured, correlated, or patterned characteristics. Consequently, a well-designed hash function must remain effective not only under random inputs but also under structured input distributions. Robustness to structured input patterns refers to the ability of a hash function to maintain near-uniform output distribution and low collision probability when inputs possess predictable regularities. This property is essential for ensuring stable performance in practical systems.

In realistic environments, keys often follow systematic patterns rather than arbitrary distributions. Sequential integers, monotonically increasing timestamps, aligned memory addresses, formatted identifiers, and strings sharing common prefixes are typical examples. For instance, identifiers such as "student_001," "student_002," and "student_003," or course codes like "CISC179-Fall2026-001," differ only in small, structured components. Similarly, database record IDs and log sequence numbers frequently increase in arithmetic progression. When a hash function is designed with only random inputs in mind, such structured datasets may interact with weaknesses in its arithmetic or bit-level design, leading to clustering and elevated collision rates.

The vulnerability arises when input regularities align with linear or insufficiently mixed hash computations. A simple modular hash function of the form $h(k) = k \bmod m$ illustrates this problem clearly. If the keys are multiples of m, every key maps to the same bucket. More generally, linear hash functions such as $h(k) = ak + b$ preserve arithmetic progressions; structured sequences in the input translate into structured sequences in the output. In such cases, the distribution may appear acceptable under random testing but degrade dramatically under patterned inputs. The issue is not randomness per se but correlation between the structure of the data and the mathematical structure of the hash function.

Suppose the hash function is defined as $h(k) = k \bmod 100$. Consider the keys 100, 200, 300, 400, and 500. These keys are not random; they follow a structured pattern—specifically, they are all multiples of 100. However, the fundamental problem is not merely the presence of structure in the input. Rather, the issue arises because this structure aligns exactly with the modulus used in the hash function.

Since each key is divisible by 100, we obtain: $100 \bmod 100 = 0$, $200 \bmod 100 = 0$, $300 \bmod 100 = 0$ and so on.

Consequently, every key maps to the same hash value (0), producing complete collision. This example illustrates how a hash function can fail when structured input patterns correlate directly with its internal arithmetic design.

To achieve robustness, a hash function must incorporate strong mixing mechanisms that diffuse input regularities across the output space. Effective mixing ensures that even highly correlated inputs produce outputs that appear statistically independent. Multiplicative hashing with carefully selected constants, combined with bitwise shifts, XOR operations, and rotations, helps break linear relationships and spread entropy from both high-order and low-order bits. This consideration is particularly important when hash table sizes are powers of two, since Naive masking — the practice of computing a hash index by simply retaining the lower-order bits of a key through a bitwise AND operation with $m - 1$ can discard meaningful variation contained in the higher-order bits, thereby increasing the risk of systematic clustering when structural entropy resides outside the preserved portion. If structured inputs differ primarily in those higher bits, a weak hash may collapse them into the same buckets. Robust design therefore requires that all portions of the input meaningfully influence the final hash value.

Naive masking

String hashing introduces additional challenges because many real-world strings exhibit substantial structural similarity, particularly in the form of long shared prefixes. Examples include file paths, URLs, and systematically generated identifiers, which often differ only in their suffixes. If a hash function disproportionately emphasizes early characters in the string, inputs that share common prefixes may map to nearby or identical hash values, resulting in clustering.

Well-designed string hashing methods, such as polynomial rolling hashes, mitigate this issue by assigning positional weights to characters and incorporating each symbol into a cumulative computation. This approach ensures that differences in later positions meaningfully influence the final hash value. Careful selection of parameters—such as the base and modulus—further reduces correlation effects and improves distribution quality.

The central objective in string hashing is to prevent localized similarity in the input (e.g., shared prefixes or minor variations) from translating into localized similarity in the output.

From a theoretical perspective, robustness to structured input patterns is closely related to universal hashing and independence properties. A hash family that approximates uniform distribution for any distinct pair of inputs limits the probability of collision to approximately $1/m$, where m is the number of buckets. Such probabilistic guarantees reduce the likelihood that any particular structured dataset can systematically exploit weaknesses in the hash design. In adversarial settings, where inputs may be intentionally crafted to cause collisions, robustness becomes even more critical to prevent degradation from expected $O(1)$ performance toward worst-case $O(n)$ behavior.

The practical implications of insufficient robustness are substantial. When a hash function fails to distribute structured inputs evenly, performance degradation follows regardless of the specific collision-resolution strategy employed. Uneven distribution increases the frequency of collisions, raises access costs, and undermines the expected constant-time behavior commonly associated with hashing. In large-scale or distributed environments, poor hashing can result in load imbalance, where certain storage locations or processing units become disproportionately burdened. In security-sensitive contexts, predictable collision patterns may be exploited to intentionally degrade system performance, potentially leading to denial-of-service scenarios. For these reasons, modern programming languages, libraries, and database systems incorporate carefully engineered mixing techniques to ensure resilience against structured and adversarial input patterns.

It is important to distinguish robustness from the avalanche effect. The avalanche effect ensures that a small change in input—such as flipping a single bit—produces widespread changes in the output bits. Robustness to structured input patterns, by contrast, concerns the global statistical behavior of the hash function when confronted with correlated or patterned datasets. Avalanche describes local sensitivity; robustness describes stability under structured distributions. Both properties are necessary

for reliable hash performance.

In summary, while theoretical analysis often relies on uniform randomness assumptions, practical hash design must anticipate structured, correlated, and predictable inputs. Robustness to structured input patterns ensures that such regularities do not undermine uniform distribution, low collision probability, and expected constant-time performance. Through strong mixing, avoidance of linear dependence, incorporation of all input bits, and probabilistic safeguards against correlation, a well-designed hash function maintains stable and reliable behavior even under highly structured workloads.

5.2.6 Computational Efficiency of the Hash Function

Computational efficiency is a fundamental design requirement of any hash function because hashing is not an occasional operation—it is performed repeatedly and often at very high frequency in practical systems. Every insertion, lookup, deletion, indexing operation, dictionary access, symbol-table query, cache lookup, or database retrieval typically begins with computing a hash value. If the hash computation itself is slow, then even a theoretically well-designed hash table will suffer poor overall performance. Thus, before discussing distribution quality, collision resistance, or avalanche behavior, one must first ensure that the hash function is computationally efficient.

In algorithmic terms, the total time of a hash-table operation can be written as

$$T_{\text{operation}} = T_{\text{hash}}(n) + T_{\text{collision handling}}$$

where n refers to the length (or size) of the key being hashed, not the number of elements in the hash table.

For example:
- If the key is an integer, then n is effectively constant (e.g., 4 or 8 bytes).
- If the key is a string of length 20, then $n = 20$.
- If the key is a long document of 10,000 characters, then $n = 10{,}000$.

Even if collision handling is expected to be $O(1)$ under standard assumptions, the hashing cost $T_{\text{hash}}(n)$ is unavoidable. Therefore, the efficiency of the hash function directly influences system throughput. When millions of keys are processed per second—as in compilers, databases, distributed systems, and networking infrastructure—the constant factors inside the hash computation become critically important.

The computational efficiency of a hash function refers to the computational resources required to evaluate the hash value of an input of length n, where time is the primary concern and space plays a comparatively minor role. In algorithmic analysis, efficiency is typically described using asymptotic notation. If the input key is modeled as a sequence of bytes

$$x = (x_0, x_1, \dots, x_{n-1})$$

then any hash function $h(x)$ must examine each component at least once in order to produce a value that depends on the entire input. Consequently, every practical hash function is subject to a fundamental lower bound

$$T(n) = \Omega(n)$$

where Ω (the Greek letter Omega) denotes a lower bound in asymptotic analysis. Since no algorithm can compute a function of all input bits without first reading them, the running time cannot grow more slowly than linearly in the input length. This establishes a theoretical minimum: linear time is unavoidable.

The central objective in hash function design is therefore to achieve linear time complexity while minimizing the associated constant factor, ensuring that the running time is bounded above by a linear function of the input size. Formally, this requirement is expressed as

$$T(n) = O(n) \tag{5.21}$$

where the notation $O(n)$ denotes an asymptotic upper bound on growth rate of the running time. Specifically, it indicates that the execution time does not increase faster than linearly as n becomes large.

Equation 5.21 can be written precisely as:

$$T(n) = an + b$$

where $a > 0$ represents the multiplicative constant corresponding to the cost incurred per input element, and $b \geq 0$ denotes the additive constant accounting for fixed overhead independent of n.

To formalize this expression using Big-O notation, the equality is converted into an inequality of the form:

$$T(n) \leq c \cdot n + b$$

for some constant $c > 0$. This inequality captures the idea that the running time is bounded above by a linear function of the input size. The constant c serves as an upper bound on the number of elementary operations required per unit of input, while the additive term b reflects fixed computational costs that do not scale with n.

For sufficiently large n, the linear term dominates the additive constant, and constant multiplicative factors do not alter the order of growth. Hence, both the coefficient of n and the fixed constant term are absorbed in the asymptotic representation, yielding the classification $T(n) = O(n)$.

The computational efficiency of a hash function is fundamentally governed by the necessity of examining each component of the input. This requirement establishes a lower bound of $\Omega(n)$, while careful algorithmic design ensures a corresponding upper bound of $O(n)$. Together, these bounds yield a tight asymptotic characterization $T(n) = \Theta(n)$, indicating that linear time is both unavoidable and optimal.

Although asymptotic notation abstracts away constant factors and additive terms, practical performance remains sensitive to these implementation details. Consequently, the design of efficient hash functions focuses not only on achieving linear growth in theory but also on minimizing the constant cost per input element in practice. An effective hash function therefore balances theoretical optimality with engineering efficiency, ensuring that hash computation does not become a bottleneck in higher-level data structure operations.

This understanding of computational efficiency provides a foundation for subsequent design considerations, including distribution quality, mixing behavior, and robustness under structured inputs.

5.3 Principles of Compression Function Design

In addition to the modulo operator, several alternative techniques exist for computing a valid array index from a hash value, depending on the implementation strategy and performance considerations of the hash table.

Bit Masking with Power-of-Two Table Size

When the array length of a hash table is chosen to be a power of two (e.g., 2^k), where k denotes the exponent such that the array length is a power of two. The computation of the array index can be efficiently performed using a bitwise masking operation instead of the modulo operator. In this approach, the index is calculated as:

$$\text{index} = \text{hash}(key) \ \& \ (\text{array_length} - 1) \tag{5.22}$$

Here, the symbol $\&$ denotes the bitwise AND operation. If the array length is 2^k, then array_length $- 1$ produces a binary number consisting of k consecutive 1s. The bitwise AND operation effectively retains only the lowest k bits of the hash value, thereby restricting the result to the valid index range $0 \leq$ index $<$ array_length.

This method is computationally efficient because bitwise operations are generally faster than division-based modulo operations. Consequently, many modern hash table implementations adopt power-of-two table sizes and use bit masking as a performance optimization. However, this technique places greater importance on the quality of the hash function, since poor distribution in the lower bits of the hash value may lead to clustering and reduced performance.

The Equation 5.22 gives the following index.

```
index = 66486 bitwise AND (16-1) → where the array length is 16 (i.e., 2^4,
↪  so k = 4).
```